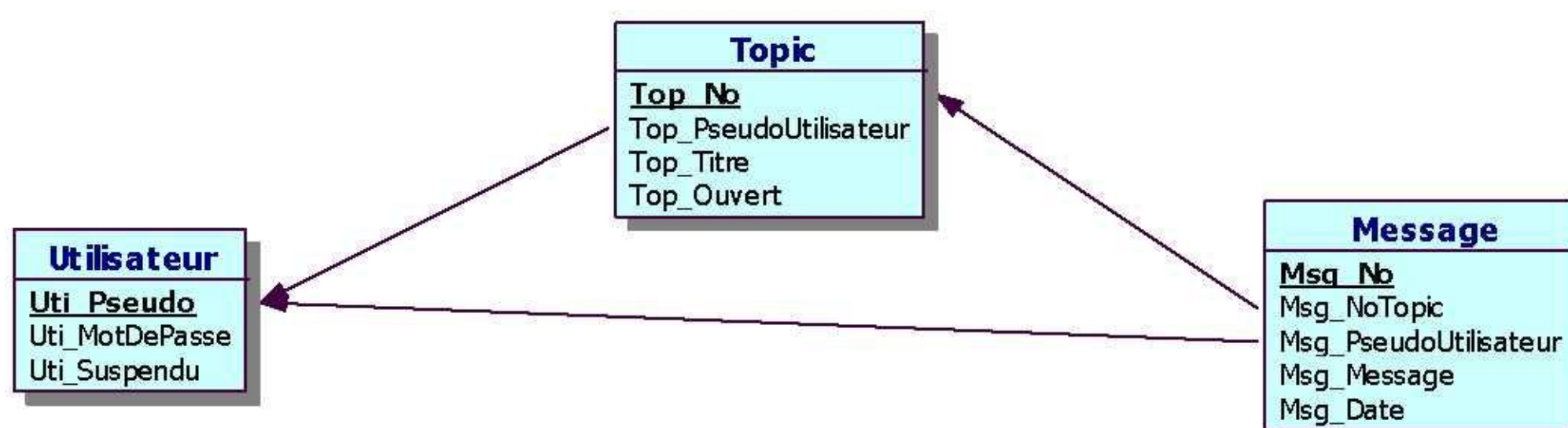


# Chapitre 8 – SQL Définir les données

|                                               |   |
|-----------------------------------------------|---|
| I. Créer une base.....                        | 1 |
| II. Supprimer une base.....                   | 1 |
| III. Utiliser une base .....                  | 1 |
| IV. Créer une table .....                     | 2 |
| V. Créer une Vue.....                         | 5 |
| VI. Supprimer une table .....                 | 5 |
| VII. Supprimer une vue.....                   | 6 |
| VIII. Modifier une table (sa structure) ..... | 6 |

Le langage de définition des données (LDD) permet de créer (*CREATE*), modifier (*ALTER*) et supprimer (*DROP*) 3 types d'objets d'une base de données (*DATABASE*) : Les tables (*TABLE*), les vues (*VIEW*) et les contraintes (*CONSTRAINT*).

Les exemples porteront sur la base *Forum* :



## I. Créer une base

Syntaxe :

```
CREATE DATABASE NomBaseDeDonnées
```

Cette primitive crée une base de données (vide) dans le SGBDR.

**Exemple** : Créer la base *Forum* :

```
Create database Forum;
```

## II. Supprimer une base

Syntaxe :

```
DROP DATABASE NomBaseDeDonnées
```

Cette primitive supprime tous les objets de la base de données (tables, vues, ...), sans confirmation.

**Exemple** : Supprimer la base *Forum* :

```
Drop database Forum;
```

## III. Utiliser une base

Syntaxe :

```
USE NomBaseDeDonnées
```



Cette primitive sélectionne la base de données sur laquelle exécuter les requêtes suivantes. Après cette commande, les tables de cette base sont référencées par leur nom directement.

Sans la commande *USE* ou bien pour utiliser des tables hors de cette base de données (rare), les noms des tables doivent être préfixés par le nom de la base.

**Exemple** : Utiliser la base *Forum* :

```
Use Forum;
```

## IV. Créer une table

### 1. Types de données

Les types de données ont une correspondance en SQL :

| Type Donnée       | Type SQL           | Remarque                                                                     |
|-------------------|--------------------|------------------------------------------------------------------------------|
| Chaîne (Longueur) | VarChar (Longueur) | Limité à 2000 caractères                                                     |
| Entier            | Integer            | Stocké sur 4 octets                                                          |
| Réel              | Real               |                                                                              |
| Date              | Date               | Contient une date avec heure, minute et seconde                              |
| Booléen           | boolean            | Converti en entier sur un octet ( <i>tinyint</i> )                           |
| Numéro Auto       | Integer            | Le numéro automatique est un attribut supplémentaire : <i>Auto_increment</i> |

### 2. Utilisation courante

Syntaxe :

```
CREATE TABLE NomTable (  
    DéfinitionChamp1,  
    DéfinitionChamp2,  
  
    ...  
    [ DéfinitionCléPrimaire ] )
```

Cette primitive crée une table nommée *NomTable*, composée des champs indiqués, et éventuellement de la clé primaire spécifiée.

### 3. Définition de chaque champ

*DéfinitionChamp* suit la syntaxe : *NomChamp* type [not null] [default valeur]

- **Type** est l'un de ceux du tableau précédent.
- **not null** rend la colonne obligatoire à la saisie. Sans not null, le champ est facultatif.
- **default valeur** donne une valeur par défaut à la colonne lors des ajouts de lignes (INSERT)

### 4. Définition de la clé primaire

*DéfinitionCléPrimaire*, qui est facultatif, s'écrit [**Constraint** *NomContrainte*] **primary key** (*ChampCléPrimaire*, ...)

**Exemple** : Créer la table *Section*



Soit

```
Create Table Section (  
    Sect_No integer auto_increment not null,  
    Sect_Nom varchar(20) not null,
```

Soit

```
Create Table Section (  
    Sect_No integer auto_increment not null,  
    Sect_Nom varchar(20) not null,
```



constraint PK\_Section primary key (Sect\_No));

primary key (Sect\_No));

### Exercice

Créer la table *Etudiant*

### Correction

Soit

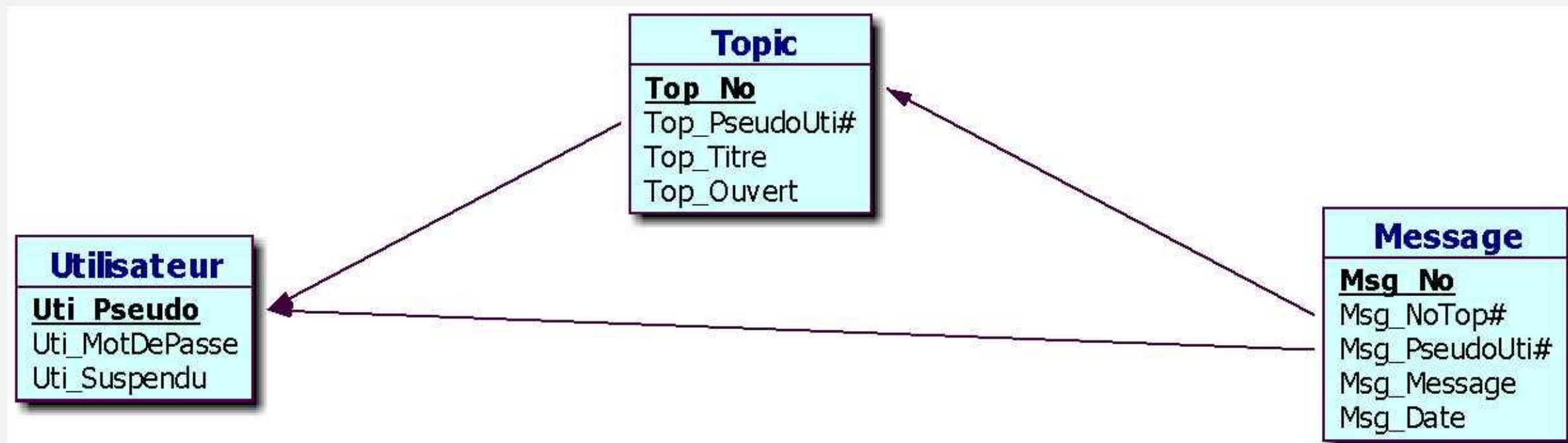
```
Create Table Etudiant (  
    Etu_No integer auto_increment not null,  
    Etu_NoSect integer not null,  
    Etu_Nom varchar(20) not null,  
    Etu_Prenom varchar(20) not null,  
    Etu_DateNais date not null,  
    constraint PK_Etudiant primary key (Etu_No));
```

Soit

```
Create Table Etudiant (  
    Etu_No integer auto_increment not null,  
    Etu_NoSect integer not null,  
    Etu_Nom varchar(20) not null,  
    Etu_Prenom varchar(20) not null,  
    Etu_DateNais date not null,  
    primary key (Etu_No));
```

### Exercice

Créer les tables *Topic*, *Utilisateur*, *Message*.



### Correction

```
Create table Utilisateur (  
    Uti_No integer auto_increment not null,           // Pour permettre un changement éventuel de pseudo  
    Uti_Pseudo varchar(20) not null,                 // Créer un index unique sur Uti_Pseudo  
    Uti_MotDePasse varchar(20) not null,  
    Uti_Suspendu boolean not null default false,  
    constraint PK_Utilisateur primary key (Uti_No));
```

```
Create table Topic (  
    Top_No integer auto_increment not null,  
    Top_NoUtilisateur integer not null,  
    Top_Titre varchar(200) not null,  
    Top_Ouvert boolean not null default true,  
    constraint PK_Topic primary key (Top_No));
```

```
Create table Message (  
    Msg_No integer auto_increment not null,  
    Msg_NoTopic integer not null,  
    Msg_NoUtilisateur integer not null,  
    Msg_Message varchar (400) not null,  
    Msg_Date datetime not null default current_timestamp,  
    constraint PK_Message primary key (Msg_No));
```



## 5. Créer une clé étrangère

Créer une clé étrangère dans une table n'est possible que si la clé primaire associée existe déjà. C'est pourquoi on préfère généralement créer toutes les tables avec seulement les clés primaires (CREATE TABLE), puis modifier ces tables pour ajouter les clés étrangères (ALTER TABLE *NomTable* Add Constraint ...).

Syntaxe :

[**Constraint** *NomContrainte*] **Foreign key** (*ChampCléEtrangère*, ...) **References** *TableParent* (*ChampCléPrimaire*, ...)

**Exemple** : Créer la table *Etudiant* avec la contrainte d'intégrité



```
Create Table Etudiant (
    Etu_No integer auto_increment not null,
    Etu_NoSect integer not null,
    Etu_Nom varchar(20) not null,
    Etu_Prenom varchar(20) not null,
    Etu_DateNais date not null,
    constraint PK_Etudiant primary key (Etu_No),
    constraint FK1_Etudiant foreign key (Etu_NoSect) references Section (Sect_No));
```

### Exercice

Compléter les créations de tables *Topic*, *Utilisateur*, *Message* pour y intégrer les clés étrangères.

### Correction

```
Create table Sujets (
    ...
    constraint PK_Sujets primary key (SujId),
    constraint FK1_Sujets foreign key (SujIdUtilisateur) references Utilisateurs (UtilId));
```

```
Create table Messages (
    ...
    constraint PK_Messages primary key (MsgId),
    constraint FK1_Messages foreign key (MsgIdSujet) references Sujets (SujId),
    constraint FK2_Messages foreign key (MsgIdUtilisateur) references Utilisateurs (UtilId));
```

## 6. Mise à jour et suppression en cascade

La contrainte d'intégrité sur une clé étrangère agit sur la table fille, mais aussi sur la table parent : Par défaut, dans le cas ci-dessus, une section ne peut être supprimée tant qu'un étudiant s'y trouve. On peut modifier ce comportement, et faire que la suppression d'une section entraîne la suppression de ses étudiants : On ajoute alors *On Delete cascade* à la fin de la définition de la contrainte.

*On Update cascade* fonctionne sur le même principe, mais il s'agit ici de la modification de la clé primaire, ce qui ne doit normalement pas se produire ...

**Exemple** : Créer la table *Etudiant* avec la contrainte d'intégrité (suppression en cascade)





Create Table Etudiant (

- Etu\_No integer auto\_increment not null,
- Etu\_NoSect integer not null,
- Etu\_Nom varchar(20) not null,
- Etu\_Prénom varchar (20) not null,
- Etu\_DateNais date not null,
- constraint PK\_Etudiant primary key (Etu\_No)
- constraint FK1\_Etudiant foreign key (Etu\_NoSect) references Section (Sect\_No) on delete cascade);

## V. Créer une Vue

Une vue est une table virtuelle, qui repose sur une requête. Elle permet de masquer certaines colonnes (inutiles ou confidentielles) voire certaines lignes. Les requêtes d'Access sont équivalentes.

Syntaxe :

CREATE VIEW *NomVue* [(*Champ1*, *Champ2*, ...)] As Select ...

**Exemple** : Créer la vue *EtudiantsSIO* avec les noms, prénoms et sections



Create View EtudiantsSIO As

- Select Etu\_Nom, Etu\_Prénom, Sect\_Nom
- From Etudiant, Section
- Where ( Sect\_Nom="SIO1" or Sect\_Nom="SIO2" )
- and Sect\_No=EtuNoSect#

## VI. Supprimer une table

Syntaxe :

DROP TABLE *NomTable*

Cette commande supprime (définitivement et sans confirmation) une table nommée *NomTable* (avec ses données).

**Exemple** : Supprimer la table *Etudiant*



Drop Table Etudiant



## VII. Supprimer une vue

Syntaxe :

DROP VIEW *NomVue*

Cette commande supprime (définitivement et sans confirmation) une vue nommée *NomVue*.

**Exemple** : Supprimer la vue *EtudiantsSIO* créée auparavant

Drop View EtudiantsSIO

## VIII. Modifier une table (sa structure)

Ajouter un champ ou une contrainte, supprimer un champ ou une contrainte

### 1. Ajouter une colonne

Syntaxe :

ALTER TABLE *NomTable* **Add** [**Column**] *DéfinitionChamp*

Où *DéfinitionChamp* est du genre *NomChamp* type [not null] [default valeur]

### 2. Ajouter une contrainte d'intégrité

Syntaxe :

ALTER TABLE *NomTable* **Add** *DéfinitionContrainte*

*DéfinitionContrainte* est du genre **Constraint** [*NomContrainte*] **Foreign key** *ChampCléEtrangère* **References** *TableParent* (*ChampCléPrimaire*)

### 3. Ajouter une clé primaire

Syntaxe :

ALTER TABLE *NomTable* **Add Primary Key** (*ChampCléPrimaire*, ...)

### 4. Supprimer un champ

Syntaxe :

ALTER TABLE *NomTable* **Drop** *NomChamp*

### 5. Supprimer une contrainte d'intégrité

Syntaxe :

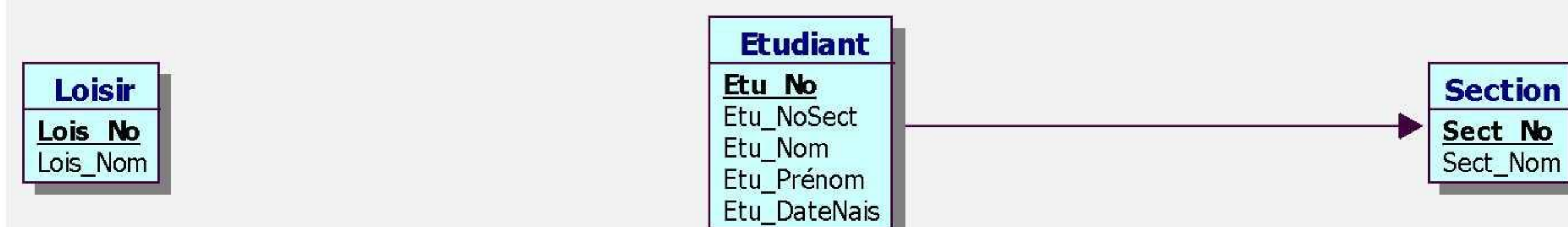
ALTER TABLE *NomTable* **Drop Foreign Key** *ChampCléEtrangère*

### 6. Supprimer une clé primaire

Syntaxe :

ALTER TABLE *NomTable* **Drop Primary Key**

**Exemple** : Ajouter le champ *Etu\_NoLoisir* dans la table *Etudiant* puis la contrainte d'intégrité associée





Alter Table Etudiant

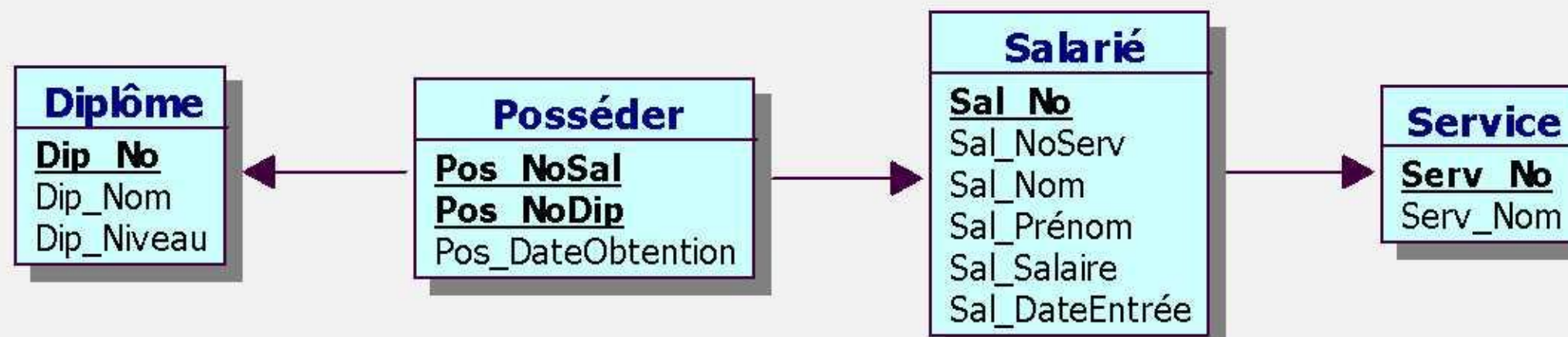
Add Etu\_NoLoisir integer;

Alter Table Etudiant

Add Constraint FK2\_Etudiant foreign key (Etu\_NoLoisir) references Loisir (Lois\_No);

## Exercice

Créer la base suivante



Correction

Create database personnel;

Use personnel;

Create table Salarié (

Sal\_No counter not null,  
Sal\_NoServ integer not null,  
Sal\_Nom text(20) not null,  
Sal\_Prénom text(20) not null,  
Sal\_Salaire money not null,  
Sal\_DateEntrée date not null,  
constraint PK\_Salarié primary key (Sal\_No));

Create table Service (

Serv\_No counter not null,  
Serv\_Nom text(20) not null,  
constraint PK\_Service primary key (Serv\_No));

Create table Diplôme (

Dip\_No counter not null,  
Dip\_Nom text(20) not null,  
Dip\_Niveau integer not null,  
constraint PK\_Diplôme primary key (Dip\_No));

Create table Posséder (

Pos\_NoSal integer not null,  
Pos\_NoDip integer not null,  
Pos\_DateObtention date not null,  
constraint PK\_Posséder primary key (Pos\_NoSal, Pos\_NoDip));

Alter table Salarié add constraint FK1\_Salarié foreign key (Sal\_NoServ) references Service (Serv\_No);

Alter table Posséder add constraint FK1\_Posséder foreign key (Pos\_NoSal) references Salarié (Sal\_No);

Alter table Posséder add constraint FK2\_Posséder foreign key (Pos\_NoDip) references Diplôme (Dip\_No);